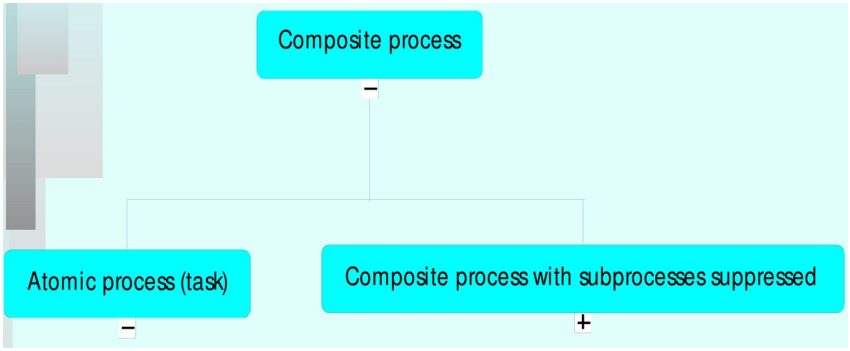
第二章 需求确定

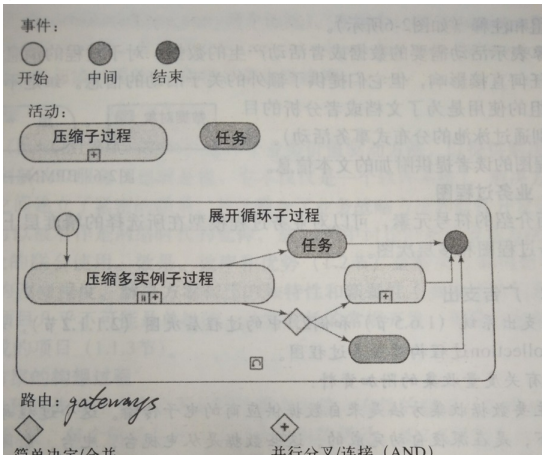
2018年9月11日　10:07

- 本章及后续章节从软件生命周期的阶段分布讲解，本章讲解需求分析阶段（包括需求确定+需求规格说明）中的需求确定阶段

1. 从业务过程到解决方案的构想
    a. 过程层次建模Process hierarchy modeling
        - BPMN中，过程可能包含其他过程（子过程），将过程中的原子活动称为任务
        i. 业务过程模型：多个不同大小的过程来定义，从全企业范围的过程到单独一个人完成的过程。
            1) 过去用数据流图作为一种复杂的功能分解技术
            2) 现在用过程层次图——不是BPMN的一部分
                - 定义了业务过程模型的静态数据结构。它显示过程的层次结构，将顶层业务过程分解为子过程。
        ii. 过程和过程分解
            1) 一个 业务过程能被看作手工操作的活动或者自动化服务
            2) 一个过程至少一个输入流和一个输出流
            3) 一个过程可以是原子的和复合的



    b. 业务过程建模BPMN——描述过程内部细节，目的是为业务人员和IT人士提供一种共同的沟通语言。
        i. 4种基本建模元素
            1) 流对象——BPMN的核心元素
                a) events事件
                b) activity活动
                    ◇ 一个子过程在圆角矩形的下边界上有一个+，表示其为一个复合活动。复合活动可分解为一组子活动，他被看作展开的子活动。其他符号决定了其他属性：循环或者多实例执行。
                c) gateway路由（网关）



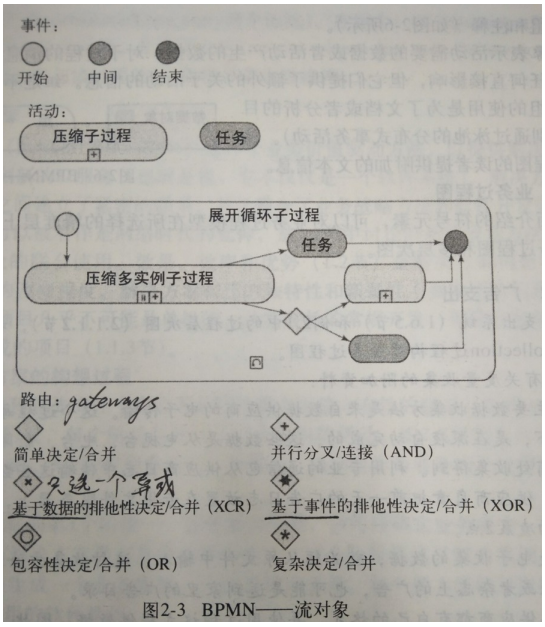


图2-3 BPMN——流对象

2) 连接对象connector
   - 消息流在连接对象中，注意选择题

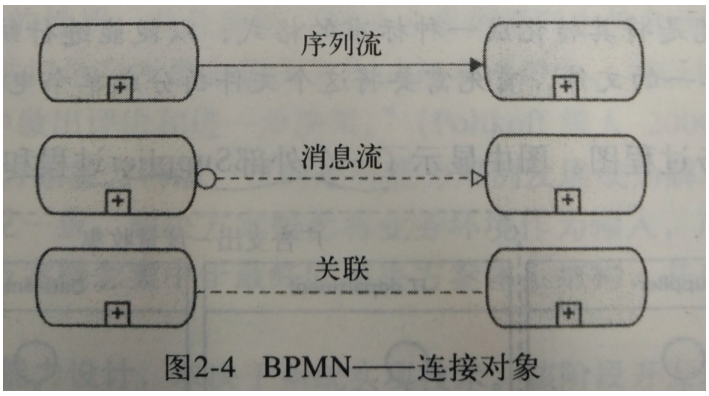
a)


图2-4 BPMN——连接对象

3) 泳池pool（泳道swimlanes）——描述单一的参与者（实体）的活动
   a) 水平的，竖直的，选择哪一种取决于作图的方便程度
4) 人工制品
   a) 如数据对象，表示活动需要的数据或者活动产生的数据。对于过程的消息流或者序列流，他们不会产生任何直接影响，但会提供额外的关于活动的信息。
   b) 组，目的是文档或分析
   c) 注释，为读者提供附加的文本信息。

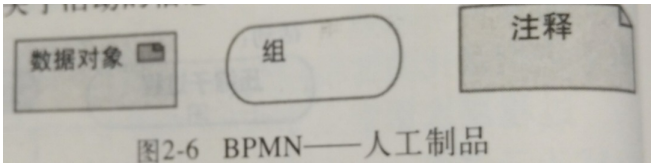
d)


图2-6 BPMN——人工制品

c. 解决方案构想solution envisioning
   - 是一个业务价值驱动方法，以提供解决当前业务问题和促进将来业务创新的IT服务。它在业务和IT利益相关者之间建立了紧密的联系，并且整合了业务战略方法和软件开发能力。

i. 关注点：
   1) 有效effectiveness
   2) 效率efficiency
   3) 边界edge

ii. 构想过程阶段
   1) 业务能力探索
      a) 这里的业务能力可理解为为企业的IT解决方案提交具体成果的能力

ii. 构想过程阶段

需求分析
1) 业务能力探索
   a) 这里的业务能力可理解为为企业的IT解决方案提交具体成果的能力
   b) 该阶段描述一个**能力案例**（确定一项功能商业价值的建模元素），即提供了解决方案思路，为每个能力生成一个业务案例
2) 解决方案能力构想
   a) 该阶段将能力用例发展成解决方案概念，确保利益相关者对这个方案的意见一致。解决方案概念将业务环境作为输入，产生的未来新工作方法的构想作为输出。
3) 软件能力设计——软件建模过程
   a) 该阶段开发软件能力体系结构，也是建模输出产品

iii. 实现策略和能力体系结构
1) 三种实现策略：
   a) 常规开发——手工、单机、从0开发软件
   b) 基于包的开发——定制先前存在的软件包
   c) 基于构件的开发（搭积木）——通过集成多个软件构件

2. 需求引导——最不需要技术，但最需要技巧的环节
a. 功能性需求——由系统期望的服务（服务陈述），侧重功能性需求
i. 分为几组：
1) 系统的范围
2) 必要的业务功能
3) 数据结构
ii. 功能性需求需要从客户处获得
iii. 收集到的需求需要仔细的分析以消除重叠和矛盾，这个过程总会导致需求评审和与客户的再次协商。
b. 非功能性需求（补充需求）——系统必须遵守的约束（约束陈述），如性能、安全性
i. 分为以下几种：
1) usability**可用性**：定义使用系统的容易程度，如适当的提示，错误处理，文档帮助
2) reusability**可复用性：**在新系统开发中，重复使用之前软件系统构件的容易程度
3) reliablity**可靠性**:系统失败的频率和严重性以及系统从失效中恢复的程度有关
4) performance**性能**：通过系统响应时间、事务处理时间、资源开销、并发等级
5) efficiency效率：考虑时间效率和成本效率
6) supportability适应性（可支持性）：可理解性、可维护性、可扩展性
7) 其他约束：政策法律约束，便捷性或可交互性等要求
c. 需求引导方法
i. 传统方法——适用于目标明确，低风险的项目
1) 面谈（发现事实和聚集信息的基本技术）
   a) 结构化访谈——需要提前准备，有一个明确的日程，预先确定问题
      i) 开放问题，客户的答案无法估计
      ii) 封闭问题，客户从提供的答案简单回答是或否
      ◇ 需要非结构化面谈进行补充
   b) 非结构化（非正式）访谈：没有预定的问题和预计目的，鼓励客户说出自己的想法，如街头采访
2) 调查表questionnaires（向客户收集信息的有效方法），一般用来作为面谈的补充形式，而非替代
   a) 调查表设计应该使回答问题变得更容易，应该尽量避免开放式问题，封闭式问题的如下三种形式：多项选择multiple-choice、评价rating questions（完全同意、反对）、排序问题ranking question
   b) 当需要调查许多人的观点而他们又地理分散的时候，调查表特别有用和经济。

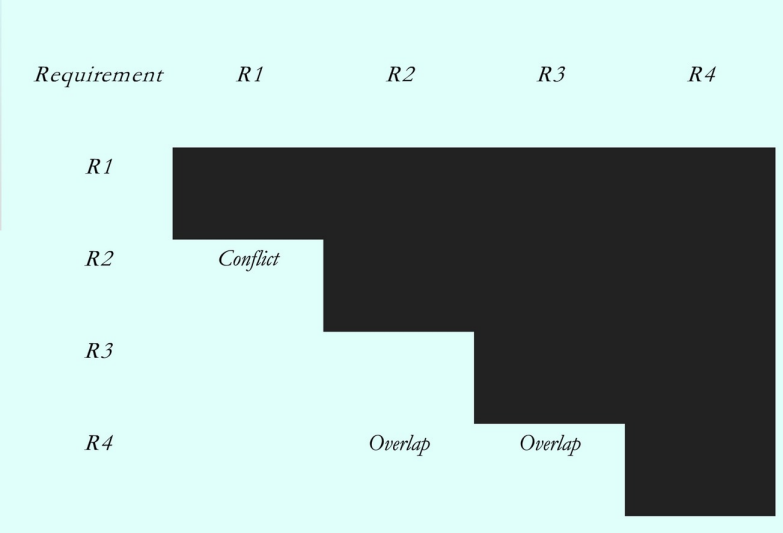
3) 观察（有效的发现事实的技术,获取面谈与调查表之外的获取不到的信息）
   a) 被动观察passive，业务分析员观察业务活动而不直接打扰或干预，比如安装摄像机
   b) 主动观察active，业务分析员参与到活动中，称为团队的一部分
   c) 解释观察explanatory，工作时，用户向观察者说明他进行的活动
4) 研究业务文档（发现用例需求和领域知识需求的宝贵技术）

ii. 现代方法——适用于风险高的项目
1) 原型法（最常使用的现代需求引导方法）quick and dirty solution to obtain feedback
   a) 丢弃原型throw away，当需求引导完成时该原型将被丢弃，主要集中在理解的最少的需求上
   b) 进化原型evolutionary，在需求引导过程之后仍被保留并用来产生最终产品。进化原型将产品发布的速度作为目标，主要集中在理解的很好的需求上
2) 头脑风暴（通过放下一系列约束来产生新思想或发现专业问题解决方案的一种会议技术）
   - 头脑风暴通过触发式问题的想法发挥作用，其他需求引导方法都不是
   a) 调解人facilitator为将产生的新想法界定问题/机会领域，这被称作问题机会陈述“probortunity”
   b) 会议最后一部，投票表决想法的优先顺序
3) 联合应用开发（joint application development,JAD）
   a) 类似于头脑风暴技术，一次会议可能需要几小时、几天或几周，参与人数25-30人
   b) 参加者：领导leader、文书scribe、客户customer（用户和经理）、开发人员developer
   c) JAD利用了群体动力优势
4) 快速应用开发（rapid application development,RAD）
   a) 不仅是一种需求引导方法，还是将软件开发作为一种过程的方法
   b) 目标时快速交付系统解决方案，技术精良程度次要
   c) 带来相应问题：
      i) 不一致的GUI设计
      ii) 支持软件复用的专业解决方案，非通用解决方案
      iii) 不足的文档
      iv) 难以维护和扩展的软件
   - 5种技术：
      i) Evolutionary prototyping
      ii) Case tools
      iii) SWAT,拥有先进工具的专业人员
      iv) Interactive JAD
      v) Timeboxing

3. 需求协商negotiation与确认validation
   - 获取的需求可能是overlap or conflict重叠的或者矛盾的，或者是ambiguous or unrealistic模棱两可或者不实际的，还可能undiscovered,out of scope,因此需要需求协商与确认
   - 需求协商与确认与需求引导同步进行
   - 需求协商以文档的草稿为基础，需求确认需要更加完整的文档确认版本

a. 超出范围的需求
   - 为了确定任何特定需求再系统范围之内还是之外，需要对照参考模型才能决定。
   - 历史上的参考模型以**关联图**的形式提供——数据流图DFD，一种流行的结构化建模技术顶层图。虽然DFD在UML中已被用例图代替，关联图仍然是建立系统边界的很好方法。

b. 需求依赖矩阵或交互矩阵N*N,表示N个需求
   - 按一种分类顺序分别再行和列的表头上列出需求标识符

- 

  - 矩阵的↗部分没有使用，剩下的单元格表示任何两个需求是否重叠、矛盾或独立。矛盾的需求应该与用户讨论，也可能要重新陈述这个需求以避免矛盾（矛盾记录保留，对后续开发可见）。重叠的需求也要重新陈述以消除重叠
  - 当需求数目较少，需求依赖矩阵是一种发现需求矛盾和重叠的简单有效的技术。

c. 需求风险和优先级
  - 解决完需求中的矛盾和重叠后，就产生了一组修正后的需求，需要对这些需求进行风险分析并排列优先级

i. 需求决定了项目的feasibility可行性，需要对风险进行优先级排序

ii. 风险分类

1) 技术风险，需求在技术上难以实现
2) 性能风险，需求实现后，会延长系统的响应时间
3) 安全风险，需求实现后，会破坏系统的安全性
4) 数据库完整性风险，需求不容易验证，并且可能导致数据的不一致性
5) 开发过程风险，需求要求开发人员采用不熟悉的非常规开发方法，如形式化规格说明方法
6) 政治风险，由于内部的政治原因，很难实现需求
7) 法律风险，需求可能触犯现行法规或者假定了法律的变更
8) 易变性风险Volatility risk，需求很可能再开发过程中不断变化或进化

4. 需求管理

a. 需求标识与分类

i. 自然语言描述需求

ii. 几种对需求进行标识分类的技术

1) 唯一标识符，由手工方式或者CASE工具的数据库生成
2) 文档层次的序列编号，考虑在需求文档中的位置的编号，如2.4.7
3) 需求分类中的顺序编号，加上一些帮助记忆的标识需求种类的名字赋值，需求种类可以实共轭能行需求、数据需求、性能需求、安全需求或其他需求
   - 每种标识方法都有赞成者与反对者。最灵活和最不易出错的方法是数据库生成唯一标识符

b. 需求层次——需求可按父子关系建立层次化结构，父级需求由子级需求组成，子集需求是父级需求有效的子需求。

c. 变更管理

i. 需要变更管理的原因：需求是变更的，在软件开发生命周期的任何阶段，需求都可能变更，可能删除已有需求或者增加新的需求。变更本身并不会导致困难，但没有管理的变更会更麻烦。开发越往前走，需求变更的开销越大。

ii. 需要强有力的政策来建立变更请求的文档，估计变更的影响并实现变更

iii. 运用的工具：需求的变更应由软件配置管理工具存储和跟踪

d. 需求可跟踪性——变更管理的一部分，或者说重要部分

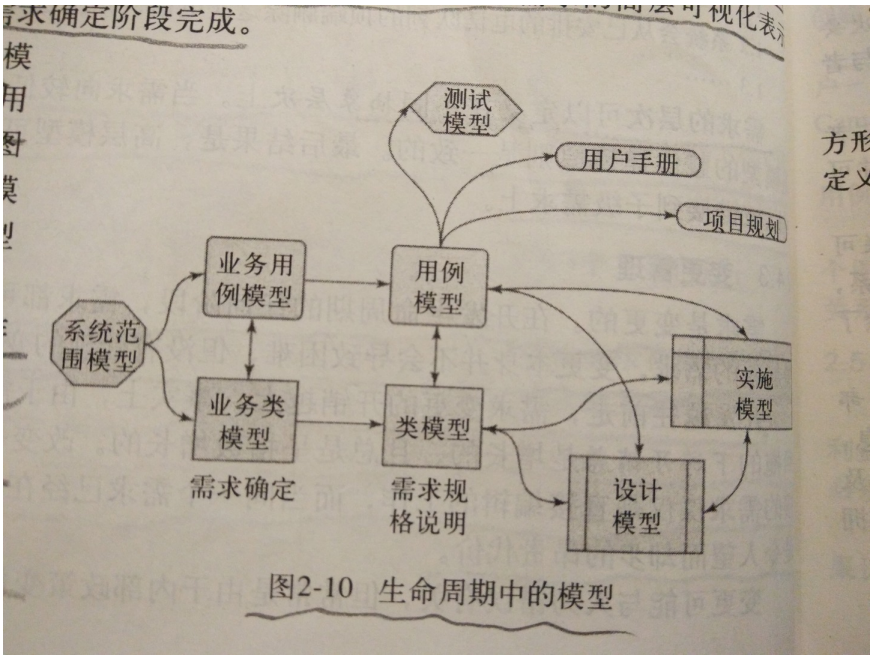
5. 需求业务模型，不属于UML，是高层可视化模型
   - 需求确定阶段：完成所收集需求的高层可视化表示（称之为业务需求建模）
   - 需求规格说明阶段：UML进行形式化的需求建模
   - 

图2-10 生命周期中的模型

a. 系统范围模型
   - 需求变更会引起系统范围蠕变，当需求中的某些变更不可避免的时候，我们必须确保请求不会超出项目可接受的范围。
   - 我们需要知道系统运行的环境。我们需要知道外部实体——期望从我们这里得到服务或为我们提供服务的其他系统、组织、人员、机器等。在业务系统中，这些服务转换成信息——数据流。
   - 因此，系统范围可以通过识别外部实体以及在外部实体和我们系统之间的输入/输出数据流来确定。

   i. UML不提供定义系统范围的好的可视化模型。因此用老式的DFD环境图context diagram表示系统范围

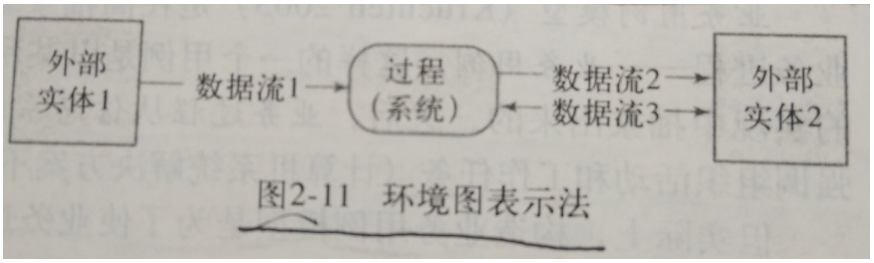
   ii. 

图2-11 环境图表示法

b. 业务用例模型——高抽象级别的用例模型
   - 业务用例模型标识高层业务进程——业务用例。这样的用例是以某种方式定义一个过程，这个过程完全是从它的实现中抽象出来的。

   i. 业务用例图，获取的是业务需求含有手工业务，即系统可能实现不了的业务服务，与用例图区分开。（整个模型代表商业观点，强调组织活动和工作任务，计算机系统不必对此提供支持）

   ii. 实际上构造业务用例模型是为了使业务过程被开发中的信息系统支持。业务用例与所谓的系统特征相对应（系统特征在视觉文档中标识。如果存在视觉文档，那么它可以代替业务用例模型）

   iii. 业务用例图的焦点是业务过程的体系结构，业务用例模型不足以向开发人员确切的表达系统应该做什么

   iv. 在需求规格说明阶段，业务用例被转换成用例。就是在此阶段标识详细的用例；

   v. 业务用例图中的业务参与者有时可以标识环境图的外部实体。这种参与者也被称为次要参与者，它们对于用例是被动的，为了和用例通信，它们需要联合主要参与者。主要参与者是系统的中心，能够与用例主动通信。主要业务参与者通过发送事件来触发用例。

   vi. 业务用例模型中重要的关系是关联关系，除了关联，一般不鼓励在业务用例之间使用UML关系

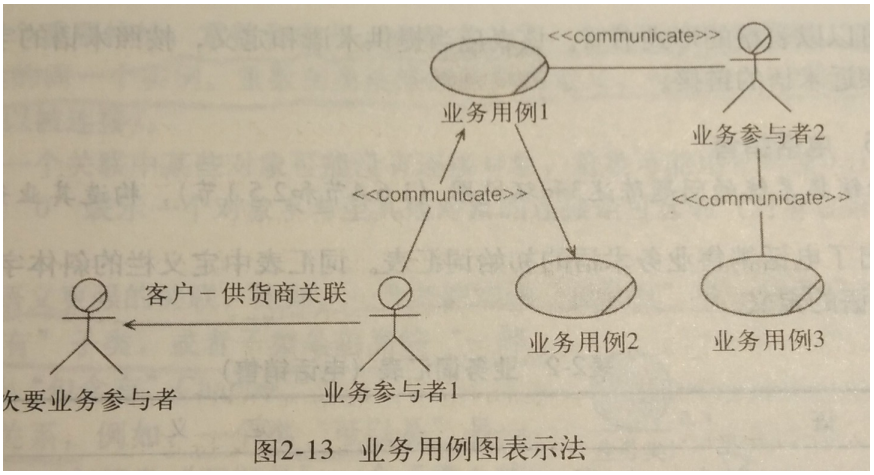


图2-13 业务用例图表示法

c. 业务词汇表，实际是业务词典
    i. 一般包含项目陈述中的名词
d. 业务类模型，和实体类模型差不多，它的抽象级别更高
    - 业务类模型是UML类模型，是标识系统中业务对象的主要类型。
    - 业务类可以通过3个UML关系连接到模型中，这三个关系是关联association、泛化generalization、聚合aggregation。关联和聚合标识类的实例（对象）之间的语义关系，泛化是类（对象类型）之间的关系。
    - 关联具有两个重要特性：重数和参与。重数定义了一个类可能的实例数，这些实例可以连接到另一个类的同一个实例。重数在类连接线的两端定义，可以是0、1或者n（这里“n”表示许多对象可以被连接）。

      为了表示在一个关联中某些对象可能没有连接对象，重数可能用一对值0..1或者0..n来表示。这对值中的“0”表示一个对象参与至其他对象的连接是可选的（对象也许有关联连接，也许没有）。

      聚合是一种语义更强的关联，比如，一个类的实例“包含有”另一个类的实例。就是说，一个超类“包含有”子类，或者子类是超类的“一部分”。例如，Book“包含有”Chapter。

      泛化是类的关系，例如，一个类“可以是”另一个类。也就是说，一个超类“可以是”一个子类，或者一个子类是“一种”超类。例如，Employee“可以是”Manager。

      非常有意思的是，经常出现这样的情况，业务用例模型的业务参与者经常被表示为业务类模型中的业务类。这与参与者对系统来说常常既是外部的又是内部的观察结果是一致的（2.5.2节）。图2-15显示了用于业务类建模的表示法。

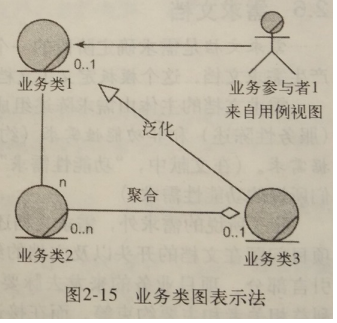
      

      图2-15 业务类图表示法

6. 需求文档
    ○ 功能性需求还可以进一步分为功能性需求和数据需求

a. 文档模板
    i. 项目准备、
        1) 项目的目的和范围
        2) 业务环境
        3) 利益相关者stackholders
        4) 多种解决方案
        5) 文档综述
    ii. 系统服务（功能需求）——占总篇幅的一半及以上
        1) 系统范围（环境图来建模）
        2) 功能性需求（业务用例图）
        3) 数据需求（没有属性方法的类图即业务类图）
    iii. 系统约束（非功能性需求）
        □ 接口需求 是系统约束

1) 界面需求，大概画出来界面长啥样，给一个界面原型
2) 性能需求，如响应时间、可靠性、吞吐量，用可测量可衡量的数据表示
3) 安全性需求，访问权限
4) 操作性需求，决定系统运行的软硬件环境
5) 政策法律需求
6) 其他约束

iv. 项目其他问题
1) 开放问题
2) 初步安排
   - 可以使用项目管理软件工具制作标准规划图，如PRET或Gantt图
3) 初步预算——初步安排的直接结果

v. 附录
1) 词汇表
   a) 术语
   b) 简写词
2) 业务文档和表格
3) 参考文献